INSTITUTO FEDERAL DE MATO GROSSO – IFMT

ATIVIDADE 04

ARTIGO TÉCNICO: ANÁLISE E APLICAÇÃO DE UMA FERRAMENTA DE OBSERVABILIDADE

Disciplina: Gerenciamento e Monitoramento de Aplicações e Infraestrutura

Curso/Turma: DevOps - 2026

Professor: Robson Keemps

Aluno: Marcelo Markus Soares Silva

Ferramenta escolhida: Grafana

Data: 2026

INSTITUTO FEDERAL DE MATO GROSSO – IFMT

Projeto de Observabilidade — Plataforma ShopFlow

Marcelo Markus Soares Silva[1]

[1] IFMT – Pós-Graduação em Engenharia DevOps – marcelo.markus@estudante.ifmt.edu.br

RESUMO

A crescente adoção de arquiteturas de microsserviços e de contêineres aumentou a complexidade operacional e tornou insuficiente a observação isolada de métricas. Este artigo analisa a ferramenta Grafana como plataforma unificada de visualização e correlação dos três pilares da observabilidade — métricas, logs e traces. A metodologia combinou revisão da documentação oficial e da literatura técnica com um experimento prático: foi construída a plataforma ShopFlow, uma aplicação de comércio eletrônico instrumentada com OpenTelemetry, executada em contêineres sobre o Podman. A telemetria foi coletada pelo OpenTelemetry Collector e armazenada no Prometheus, no Loki e no Tempo, sendo consumida pelo Grafana por meio de fontes de dados provisionadas como código. O experimento demonstrou a capacidade de correlacionar um log ao trace correspondente por meio do identificador trace_id e de refletir, em tempo quase real, uma indisponibilidade simulada: a taxa de erro alcançou cerca de 33% e o indicador de disponibilidade alternou de DISPONÍVEL para INDISPONÍVEL e de volta após a recuperação. Conclui-se que o Grafana entrega uma camada de visualização madura e agnóstica ao backend, cujo principal limite é não armazenar a telemetria, dependendo de fontes externas.

Palavras-chave: observabilidade; monitoramento; Grafana; métricas; traces distribuídos.

1 INTRODUÇÃO

A evolução dos sistemas de software para arquiteturas distribuídas e conteinerizadas trouxe flexibilidade de implantação, mas elevou a dificuldade de diagnosticar falhas. Em um ambiente de microsserviços, um único fluxo de negócio pode atravessar vários serviços, e a análise tradicional baseada apenas em logs ou em um painel isolado torna-se lenta e pouco conclusiva.

Nesse contexto, a observabilidade surge como a capacidade de responder, a partir da telemetria, a perguntas ainda não formuladas sobre o comportamento do sistema. Para operá-la, recorre-se a ferramentas de visualização que reúnam métricas, logs e traces em um único ambiente, papel em que o Grafana se destaca por ser um projeto de código aberto capaz de consultar múltiplas fontes de dados.

Este artigo analisa a ferramenta Grafana, descreve a sua arquitetura e as suas principais funcionalidades e a aplica a um experimento de observabilidade em uma aplicação de comércio eletrônico, a plataforma ShopFlow. Além desta introdução, o texto apresenta a fundamentação teórica, a análise da ferramenta, a instalação, o experimento, os resultados, as vantagens e limitações, uma comparação com outra solução e as considerações finais.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Observabilidade

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir das suas saídas os dados de telemetria. Diferente do monitoramento tradicional, centrado em métricas previamente definidas e em painéis fixos, a observabilidade pressupõe explorar os dados para investigar comportamentos não previstos. Majors, Fong-Jones e Miranda destacam três pilares que sustentam essa capacidade: métricas, logs e traces distribuídos.

2.2 Métricas, logs e traces

As métricas são séries temporais numéricas, leves e adequadas para agregações e alertas rápidos; no modelo proposto pela comunidade Prometheus, costumam seguir os princípios RED (Rate, Errors e Duration). Os logs registram eventos discretos e ricos em contexto, sendo mais úteis quando estruturados em formato legível por máquina, como o JSON. Os traces distribuídos reconstroem o caminho completo de uma requisição entre serviços, representando cada etapa como um span e associando-os por um identificador de trace.

A correlação entre os três sinais torna a investigação eficiente: um identificador de trace presente nas métricas, nos logs e nos spans permite saltar de um sintoma agregado até a causa específica. A instrumentação padronizada pelo OpenTelemetry favorece essa correlação ao propagar o contexto de trace automaticamente.

2.3 Outros conceitos relevantes

A engenharia de confiabilidade de sites (SRE) consolidou os conceitos de indicador (SLI), objetivo (SLO) e acordo (SLA) de nível de serviço, que orientam decisões com base em metas mensuráveis. Já a conteinerização tornou os serviços efêmeros e numerosos, reforçando a necessidade de telemetria padronizada e agregável.

3 FERRAMENTA ANALISADA

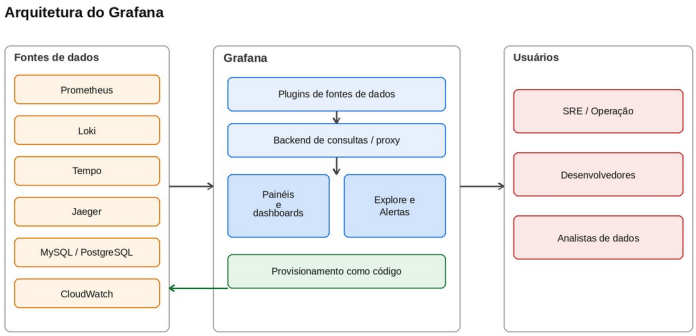
3.1 Visão geral

O Grafana é uma plataforma de código aberto, mantida pela Grafana Labs, usada para visualizar e explorar dados de telemetria. Consolidou-se como uma interface de observabilidade agnóstica ao fornecedor: em vez de armazenar os dados, conecta-se a fontes externas como Prometheus, Loki e Tempo e oferece painéis, consultas exploratórias e alertas sobre elas. Pode ser implantado como serviço gerenciado, binário ou contêiner; neste trabalho usou-se a edição de código aberto em contêiner.

3.2 Arquitetura

A arquitetura do Grafana organiza-se em torno de plugins de fonte de dados. Para cada fonte, um plugin define como as consultas são formuladas e como a resposta é convertida em séries. O front-end, escrito em JavaScript/React, renderiza os painéis, enquanto o back-end atua como proxy de consultas, mediando o acesso às fontes. Sobre essa base apoiam-se os painéis, o modo Explore, o alerta unificado e o provisionamento como código. A Figura 1 ilustra os componentes e o fluxo de dados.

Figura 1 – Arquitetura da ferramenta analisada



Fonte: Elaborado pelo autor (2026).

3.3 Principais funcionalidades

A Tabela 1 (Quadro 1) resume as funcionalidades mais relevantes do Grafana e a forma como foram empregadas no cenário analisado.

Quadro 1 – Funcionalidades do Grafana e aplicação no cenário | Fonte: Elaborado pelo autor (2026).

Funcionalidade/Recurso	Descrição	Aplicação no cenário
Dashboards	Painéis com gráficos alimentados por variáveis e transformações.	Visão RED e de negócio da ShopFlow
Explore	Consulta ad hoc às fontes sem criar painéis.	Investigação de métricas, logs e traces
Alertas unificados	Regras de alerta para diversos back-ends em um só lugar.	Base para alertas de SLO (não configurados aqui)
Correlação de sinais	Saltos entre métrica, log e trace via identificadores.	Do gráfico ao trace pelo campo trace_id
Fontes de dados	Conexão a múltiplos back-ends simultaneamente.	Prometheus, Loki e Tempo no mesmo painel
Provisionamento como código	Configuração versionada de fontes e painéis.	Datasources e dashboard em arquivos YAML/JSON

4 INSTALAÇÃO E CONFIGURAÇÃO

4.1 Ambiente utilizado

O experimento foi executado em uma estação de trabalho com Fedora 42, com SELinux em modo permissivo para contêineres, Podman 5.8 em modo rootless e Terraform 1.15. Todos os componentes Grafana, Prometheus, Loki, Tempo, OpenTelemetry Collector e a aplicação foram executados como contêineres em uma rede dedicada, sem orquestrador, o que mantém o ambiente reproduzível e leve.

4.2 Instalação

A instalação baseou-se em imagens oficiais fixadas em versões específicas: grafana/grafana, prom/prometheus, grafana/loki, grafana/tempo e otel/opentelemetry-collector-contrib. Em vez de subir os contêineres manualmente, a infraestrutura foi descrita em Terraform e aplicada com o comando terraform apply, que cria a rede, os volumes e os contêineres, e publica as portas no host. O Grafana ficou acessível na porta 3000.

4.3 Configuração

A configuração foi feita inteiramente por provisionamento, sem uso da interface. Um arquivo de fontes de dados registra as conexões com Prometheus, Loki e Tempo; um provedor de painéis carrega o dashboard a partir de um arquivo JSON versionado. As consultas dos painéis usam PromQL e TraceQL, e a correlação entre sinais é configurada por meio de enlaces que levam do identificador de trace no log até o trace no Tempo.

5 EXPERIMENTO PRÁTICO

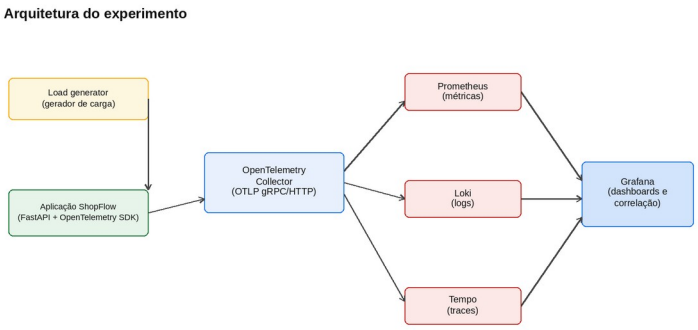
5.1 Cenário experimental

O cenário é a plataforma ShopFlow, uma API de comércio eletrônico implementada em Python com o framework FastAPI. A aplicação expõe rotas de catálogo, busca, autenticação e checkout, além de instrumentar, com o SDK OpenTelemetry, métricas de negócio e spans das operações. Um gerador de carga produz tráfego contínuo, o que mantém os painéis com dados atualizados.

5.2 Arquitetura do experimento

A aplicação envia métricas, logs e traces por OTLP ao OpenTelemetry Collector, que distribui cada sinal para o backend apropriado: Prometheus para métricas, Loki para logs e Tempo para traces. O Grafana consulta os três armazenamentos e apresenta o resultado de forma correlacionada. A Figura 2 apresenta essa disposição.

Figura 2 – Arquitetura do experimento



Fonte: Elaborado pelo autor (2026).

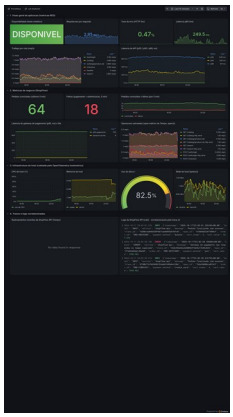
5.3 Execução

Após a implantação da stack com terraform apply, o gerador de carga foi iniciado e aguardou-se a estabilização da telemetria. Para produzir evidências de comportamento anômalo, foi acionado um script de injeção de falha que simula a indisponibilidade do serviço; em seguida, o serviço foi restabelecido. As telas foram capturadas automaticamente por um script que usa o navegador em modo headless.

5.4 Evidências

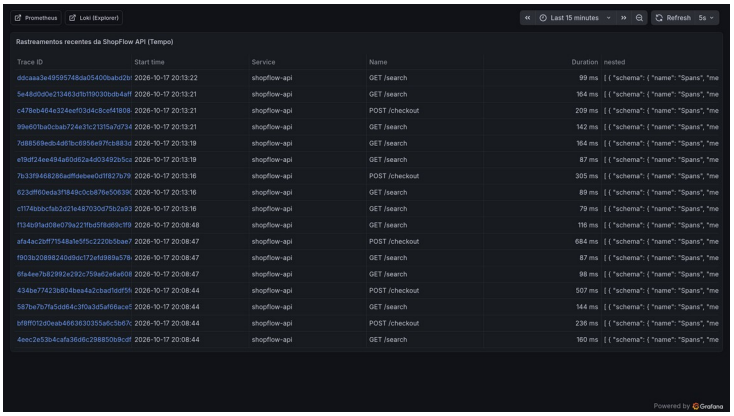
As figuras a seguir reúnem as evidências do experimento. A Figura 3 mostra o dashboard completo; a Figura 4, os traces no Tempo; a Figura 5, os logs correlacionados no Loki; e as Figuras 6 e 7, a indisponibilidade simulada e a recuperação do serviço.

Figura 3 – Dashboard de observabilidade da ShopFlow



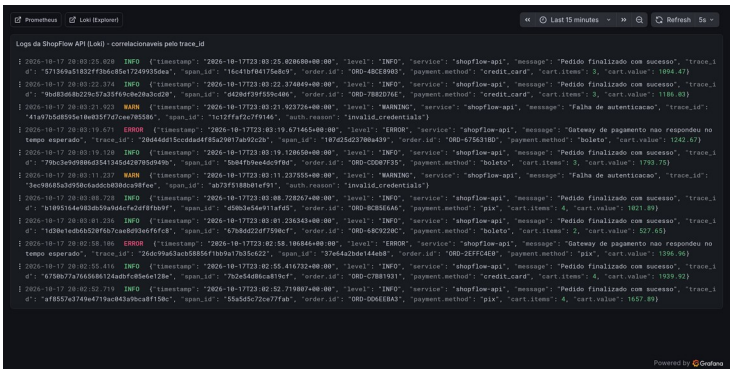
Fonte: Elaborado pelo autor (2026).

Figura 4 – Rastreamentos recentes consultados no Grafana Tempo



Fonte: Elaborado pelo autor (2026).

Figura 5 – Logs estruturados correlacionados no Grafana Loki



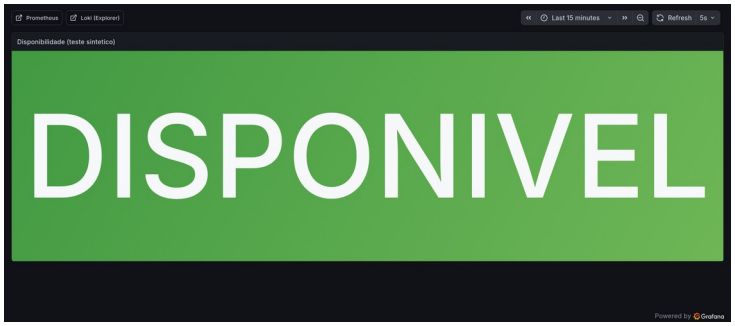
Fonte: Elaborado pelo autor (2026).

Figura 6 – Indisponibilidade simulada na ShopFlow API



Fonte: Elaborado pelo autor (2026).

Figura 7 – Recuperação do serviço após o restabelecimento



Fonte: Elaborado pelo autor (2026).

6 RESULTADOS E DISCUSSÃO

Os resultados confirmam que o Grafana consolidou, em um único ambiente, a leitura dos três pilares da observabilidade. O dashboard provisionado exibiu, de forma simultânea, a taxa de requisições, a taxa de erro, a latência por percentis e indicadores de negócio, além de métricas de infraestrutura do host, traces e logs.

Durante a falha simulada, a taxa de erro medida alcançou aproximadamente 33% e o indicador de disponibilidade mudou de DISPONÍVEL para INDISPONÍVEL, enquanto os painéis de log passaram a registrar mensagens de erro. Após o restabelecimento, os indicadores retornaram ao estado normal. Esse comportamento evidencia a utilidade prática da visualização para detectar e acompanhar incidentes.

A correlação entre sinais mostrou-se o recurso mais valioso para o diagnóstico: a partir de um log de erro foi possível acessar diretamente o trace correspondente pelo campo trace_id, reduzindo o tempo necessário para localizar a operação que falhou.

7 VANTAGENS E LIMITAÇÕES

7.1 Vantagens

Interface unificada para métricas, logs e traces, com correlação entre sinais em um único ambiente.

Agnóstico ao backend: a mesma ferramenta consulta Prometheus, Loki, Tempo e outras fontes.

Provisionamento como código, que versiona fontes de dados e dashboards e facilita areprodutibilidade.

7.2 Limitações e dificuldades

Não armazena telemetria: depende de back-ends externos para métricas, logs e traces.

Exige familiaridade com linguagens de consulta distintas, como PromQL e TraceQL.

O desempenho cai em consultas de alta cardinalidade ou em janelas de tempo muito amplas.

8 COMPARAÇÃO COM OUTRA FERRAMENTA

Compara-se o Grafana ao Kibana, uma solução de visualização associada ao Elastic Stack. Enquanto o Grafana foi projetado para ser agnóstico às fontes de dados, o Kibana é otimizado para o ecossistema Elastic. A Tabela 2 (Quadro 2) sintetiza as principais diferenças.

Quadro 2 – Comparação entre Grafana e Kibana | Fonte: Elaborado pelo autor (2026).

Característica	Grafana	Kibana
Tipo/modelo	Visualização unificada multi-fonte (open source)	Análise sobre o Elastic Stack
Métricas	Prometheus, Graphite, InfluxDB, entre outras	Elasticsearch e métricas ingeridas
Logs	Loki, Elasticsearch e outros	Elasticsearch (foco principal)
Traces	Tempo, Jaeger, X-Ray	Elastic APM
Dashboards	Painéis, variáveis e transformações	Dashboards e Lens
Alertas	Alerta unificado para todas as fontes	Alertas do Elastic Stack
Integrações	Amplo catálogo de plugins	Forte integração com o Elastic Stack
Instalação	Contêiner leve e independente	Requer cluster Elasticsearch
Outro aspecto	Agnóstico ao backend	Melhor desempenho com a stack Elastic

9 CONCLUSÃO

Este artigo analisou o Grafana como plataforma de visualização para observabilidade e o aplicou a um experimento com a plataforma ShopFlow. Constatou-se que a ferramenta cumpre bem o papel de camada unificada de visualização e correlação, especialmente por não exigir a adoção de um backend específico. O experimento demonstrou, na prática, a detecção de indisponibilidade, a leitura de indicadores RED e a correlação entre logs e traces.

Como principais limitações, destacam-se a dependência de armazenamentos externos e a curva de aprendizado das linguagens de consulta. Trabalhos futuros podem explorar a

configuração de alertas baseados em SLO e a evolução do painel para métricas de custo e de experiência do usuário.

REFERÊNCIAS

[1] SIGELMAN, B. et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.

[2] MAJORS, C.; FONG-JONES, L.; MIRANDA, G. Observability Engineering: Achieving Production Excellence. Sebastopol: O'Reilly Media, 2022.

[3] BEYER, B. et al. Site Reliability Engineering: How Google Runs Production Systems. Sebastopol: O'Reilly Media, 2016.

[4] GRAFANA LABS. Grafana Documentation. Disponível em: https://grafana.com/docs/. Acesso em: 2026.

[5] PROMETHEUS AUTHORS. Prometheus Documentation. Disponível em: https://prometheus.io/docs/. Acesso em: 2026.

[6] OPEN TELEMETRY. OpenTelemetry Documentation and Specifications. Disponível em: https://opentelemetry.io/docs/. Acesso em: 2026.

[7] SILVA, Marcelo Markus Soares. Repositório do projeto ShopFlow — Observabilidade. Disponível em: <https://github.com/marcelomarkus/IFMT-Pos-DevOps-Observabilidade>. Acesso em: out. 2026.